

模組 5：使用 Cloud Buildpacks 從 Google App Engine 遷移至 Cloud Run

來源：<https://codelabs.developers.google.com/codelabs/cloud-gae-python-migrate-5-runbldpks?hl=zh-tw>

步驟數：7

瞭解如何使用 Cloud Buildpacks 將簡易的 App Engine 應用程式容器化，並遷移至 Cloud Run。

目錄

1. [1. 總覽](#)
2. [2. 背景](#)
3. [3. 設定/準備工作](#)
4. [4. 容器化應用程式](#)
5. [5. 建構及部署](#)
6. [6. 摘要/清除](#)
7. [7. 其他資源](#)

1. 總覽

本系列程式碼研究室 (可自行調整步調的實作教學課程) 旨在協助 [Google App Engine](#) (標準) 開發人員完成一系列遷移作業，進而將應用程式現代化。完成後，使用者可以明確將應用程式容器化，以便在 [Cloud Run](#) (Google Cloud 的容器代管服務，與 App Engine 類似) 和其他容器代管服務中執行，讓應用程式更具 *可攜性*。

本教學課程將說明如何使用 [Cloud Buildpacks](#) (Docker 的替代方案)，將 App Engine 應用程式容器化，以便部署至 Cloud Run 全代管服務。Cloud Buildpacks 可將應用程式容器化，不必管理 `Dockerfile` 檔案，甚至不必瞭解 Docker。

本程式碼研究室適用於已將應用程式從原始內建服務移出，並移植到 Python 3 的 Python 2 App Engine 開發人員，他們現在希望在容器中執行應用程式。程式碼研究室會從已完成的模組 2 或模組 3 Python 3 應用程式開始。

注意：Cloud Buildpacks 僅支援 Python 3.7 以上版本 (以及 Go 1.10 以上版本、Node.js 10 以上版本、Java 8 和 11，以及 .NET Core 3.1 以上版本)，因此 Python 2 App Engine 應用程式必須先移植到 3.x，才能使用 Cloud Buildpacks。如要將 2.x 應用程式容器化，請務必使用 *Docker*。建議您暫時略過本教學課程，直接前往[第 4 模組程式碼實驗室：使用 Docker 從 Google App Engine 遷移至 Cloud Run](#)。

課程內容

- 使用 Cloud Buildpacks 將應用程式容器化
- 將容器映像檔部署至 Cloud Run

軟硬體需求

- 具有下列項目的 [Google Cloud Platform 專案](#)：
 - [有效的 GCP 帳單帳戶](#)
 - [已啟用 App Engine](#)
- Python 基礎技能
- 熟悉常見的 Linux 指令
- 具備[開發及部署](#) App Engine 應用程式的基本知識
- 建議您先完成[第 2 堂](#)或[第 3 堂](#)程式碼研究室 (包括將其移植到 Python 3)，再開始進行本堂 (第 5 堂) 程式碼研究室。
- 可正常運作的 Python 3 App Engine 應用程式，已準備好進行容器化
 - [單元 2 Cloud NDB 範例](#) 或
 - [第 3 堂課：Cloud Datastore 範例](#)

問卷調查

2. 背景

App Engine 和 Cloud Functions 等 PaaS 系統可為團隊和應用程式帶來許多便利。舉例來說，這些無伺服器平台可讓系統管理員/DevOps 專注於建構解決方案。應用程式可視需要自動擴充資源，並在用量降至零時縮減資源，搭配依用量計費的機制控管費用，且支援多種常見的開發語言。

不過，容器的彈性也很吸引人，因為您可以選擇任何語言、程式庫和二進位檔。[Google Cloud Run](#) 的目標是為使用者提供兩全其美的方案，兼具無伺服器運算的便利性與容器的彈性。

本程式碼研究室不會說明如何使用 Cloud Run，相關內容請參閱 [Cloud Run 說明文件](#)。本節的目標是讓您瞭解如何將 App Engine 應用程式容器化，以使用於 Cloud Run (或其他服務)。繼續操作前，請先瞭解幾件事，主要是使用者體驗會略有不同，因為您不再需要取得應用程式碼並部署，所以會稍微降低層級。

您需要瞭解容器的某些概念，例如如何建構及部署容器。您也可以決定要在容器映像檔中放入哪些內容，包括網頁伺服器，因為您不再使用 App Engine 的網頁伺服器。如果您不想採用這種做法，繼續使用 App Engine 也是不錯的選擇。

在本教學課程中，您將瞭解如何將應用程式容器化、移除 App Engine 設定檔，以及管理網頁伺服器 (包括如何啟動應用程式)。

這項遷移作業包含下列步驟：

1. 設定/準備工作
 2. 將應用程式容器化
 - 取代設定檔
 - 修改應用程式檔案
-

步驟 3 · 約 4 分鐘

3. 設定/準備工作

在開始本教學課程的主要部分之前，請先設定專案、取得程式碼，然後部署基準應用程式，確保我們從可運作的程式碼著手。

1. 設定專案

如果您已完成[第 2 模組](#)或[第 3 模組](#)程式碼研究室，建議重複使用該專案 (和程式碼)。或者，您也可以[建立全新專案](#)，或重複使用其他現有專案。確認專案已啟用有效的帳單帳戶和 App Engine (應用程式)。

2. 取得基準範例應用程式

本程式碼研究室的先決條件之一，是必須有可運作的第 2 或第 3 模組範例應用程式。如果沒有，建議先完成任一教學課程 (上方連結)，再繼續進行本程式碼研究室。如果您已熟悉這些內容，可以直接從下方取得其中一個程式碼資料夾。

無論您使用自己的或我們的，本教學課程都從這裡開始。本程式碼研究室會逐步說明遷移程序，完成後，程式碼應與「Module 5 FINISH」存放區資料夾中的內容大致相同。

- 開始：
 - Cloud NDB：[第 2 堂課的程式碼](#)
 - Cloud Datastore：[第 3 堂課的程式碼](#)
- 完成：[第 5 個模組的程式碼](#)
- [整個存放區](#) (複製或下載 ZIP 檔案)

START 檔案 (您的或我們的) 目錄應如下所示：

```
$ ls
README.md      main.py        templates
app.yaml       requirements.txt
```

3. (重新) 部署基準應用程式

請立即執行剩餘的準備步驟：

1. 重新熟悉 `gcloud` 指令列工具
2. 使用 `gcloud app deploy` 重新部署範例應用程式
3. 確認應用程式在 App Engine 上順利執行

順利執行這些步驟後，即可將其容器化。

4. 容器化應用程式

Docker 是當今業界的標準容器化平台，如前所述，使用這項工具的一項挑戰是，您需要費心管理有效的 `Dockerfile` (決定容器映像檔建構方式的設定檔)。另一方面，[Buildpack](#) 只需要少量工作，因為它會使用內省來判斷應用程式的依附元件，盡可能提高應用程式的 Buildpack 容器效率。

如果您想略過 Docker 相關知識，並將 App Engine 應用程式容器化，以便在 Cloud Run 或任何其他容器代管平台執行，那麼您來對地方了。如果您有興趣瞭解如何使用 Docker 進行應用程式容器化，可以在完成本程式碼研究室後，進行[第 4 模組的程式碼研究室](#)。與這個範例相同，但使用 Docker，可讓您深入瞭解如何管理容器映像檔。

遷移步驟包括取代 App Engine 設定檔，以及指定應用程式的啟動方式。下表列出各平台類型應有的設定檔。比較 App Engine 欄與 Buildpack 欄 (以及 Docker，視需要)：

說明	App Engine	Docker	建構套件
一般設定	<code>app.yaml</code>	<code>Dockerfile</code>	(<code>service.yaml</code>)
第三方程式庫	<code>requirements.txt</code>	<code>requirements.txt</code>	<code>requirements.txt</code>
第三方設定	<code>app.yaml</code> (另有 <code>appengine_config.py</code> 和 <code>lib</code> [僅限 2.x])	(不適用)	(不適用)
啟動	(不適用) 或 <code>app.yaml</code> (如果使用 <code>entrypoint</code>)	<code>Dockerfile</code>	<code>Procfile</code>
忽略檔案	<code>.gcloudignore</code> 和 <code>.gitignore</code>	<code>.gcloudignore</code> 、 <code>.gitignore</code> 和 <code>.dockerignore</code>	<code>.gcloudignore</code> 和 <code>.gitignore</code>

應用程式容器化後，即可部署至 Cloud Run。其他 Google Cloud 容器平台選項包括 [Compute Engine](#)、[GKE](#) 和 [Anthos](#)。

一般設定

App Engine 會自動啟動應用程式，但 Cloud Run 不會。[Procfile](#) 的作用與 `app.yaml entrypoint` 指令類似，在範例應用程式中，`Procfile` 會執行 `python main.py` 來啟動 Flask 開發伺服器。您也可以視需要使用 `gunicorn` 等正式版網路伺服器，但如果使用這類伺服器，請務必將其新增至 `requirements.txt`。如要進一步瞭解如何使用 Buildpacks 從原始碼部署，請參閱[這篇 Cloud Run 說明文件頁面](#)。

您只需要將 `app.yaml entrypoint` 指令移至 `Procfile` 即可。如果沒有，請暫時使用 Flask 開發伺服器，因為這只是範例測試應用程式，可讓使用者熟悉遷移作業。應用程式會是您最瞭解的特定啟動指令。在幕後，Cloud Run 服務會建立 `service.yaml`，看起來/作用方式更像是 `app.yaml`。如要查看自動產生的 `service.yaml`，請前往類似這樣的連結，但請將 `SVC_NAME` 和 `REGION` 換成您的服務：https://console.cloud.google.com/run/detail/REGION/SVC_NAME/yaml/view。

第三方程式庫

`requirements.txt` 檔案不需要變更，Flask 應會與 Datastore 用戶端程式庫 (Cloud Datastore 或 Cloud NDB) 一併存在。如要使用其他符合 WSGI 規範的 HTTP 伺服器 (例如 Gunicorn，撰寫本文時的最新版本為 20.0.4)，請將 `gunicorn==20.0.4` 新增至 `requirements.txt`。

第三方設定

Buildpack 不支援 Python 2，因此我們不會在此討論。在 Cloud Run 容器中執行的 Python 3 應用程式與 Python 3 App Engine 應用程式類似，都必須在 `requirements.txt` 中指定第三方程式庫。

啟動

Python 3 使用者可以選擇轉換 `app.yaml` 檔案，在 `handlers` 區段中採用 `entrypoint` 指令，而非 `script: auto` 指令。如果您在 Python 3 `app.yaml` 中使用 `entrypoint`，看起來會像這樣：

```
runtime: python38
entrypoint: python main.py
```

`entrypoint` 指令會告訴 App Engine 如何啟動伺服器。你可以直接將其移至 `Procfile`。以下摘要說明兩個平台之間，進入點指令的放置位置：這會直接轉換為下方內容；此外，FYI 也會顯示 Docker 對應項目：

- Buildpack： `Procfile` 中的行：`web: python main.py`
- Docker： `Dockerfile` 中的行：`ENTRYPOINT ["python", "main.py"]`

如要進行測試和暫存，只要從 Python 執行 Flask 的開發伺服器即可，就像我們在上方所做的一樣，但開發人員可以選擇更強大的工具，用於實際執行環境，例如使用 `gunicorn` 的 [Cloud Run 快速入門範例](#)。

應用程式檔案

所有第 2 模組或第 3 模組應用程式都完全相容於 Python 2-3，也就是說，`main.py` 的核心元件不會有任何變更，我們只會新增幾行啟動程式碼。在 `main.py` 底部新增一對程式碼行，啟動開發伺服器，因為 Cloud Run 需要開啟通訊埠 8080，才能呼叫您的應用程式：

```
if __name__ == '__main__':
    import os
    app.run(debug=True, threaded=True, host='0.0.0.0',
            port=int(os.environ.get('PORT', 8080)))
```

5. 建構及部署

App Engine 設定已由 Buildpack 取代，且原始碼檔案更新完成後，您就可以在 Cloud Run 上執行應用程式。在此之前，我們先簡單討論服務。

服務與應用程式

App Engine 主要用於託管應用程式，但也能託管由一系列微服務組成的網路服務或應用程式。在 Cloud Run 中，所有項目都是服務，無論是實際服務或具有網頁介面的應用程式，因此請將其使用視為服務部署，而非應用程式。

除非您的 App Engine 應用程式是由多項服務組成，否則部署應用程式時，您其實不必進行任何命名。但使用 Cloud Run 時，您需要想出服務名稱。而 App Engine 的 `appspot.com` 網域則會包含專案 ID (例如

`https://PROJECT_ID.appspot.com`)，以及可能包含地區 ID 縮寫 (例如

`http://PROJECT_ID.REGION_ID.r.appspot.com`)。

不過，Cloud Run 服務的網域會包含服務名稱、區域 ID 縮寫和雜湊，但不會包含專案 ID，例如 `https://SVC_NAME-HASH-REG_ABBR.a.run.app`。總之，請開始思考服務名稱！

部署服務

執行下列指令，建構容器映像檔並部署至 Cloud Run。系統提示時，請選取地區並允許[未經驗證的連線](#)，方便測試，然後視需要選取地區，其中 `SVC_NAME` 是您要部署的服務名稱。


```
$ gcloud run deploy SVC_NAME --source .
```

Please choose a target platform:

- [1] Cloud Run (fully managed)
- [2] Cloud Run for Anthos deployed on Google Cloud
- [3] Cloud Run for Anthos deployed on VMware
- [4] cancel

Please enter your numeric choice: 1

To specify the platform yourself, pass `--platform managed`. Or, to make this the default target platform, run `gcloud config set run/platform managed`.

Please specify a region:

- [1] asia-east1
- [2] asia-east2
- [3] asia-northeast1
- [4] asia-northeast2
- [5] asia-northeast3
- [6] asia-south1
- [7] asia-southeast1
- [8] asia-southeast2
- [9] australia-southeast1
- [10] europe-north1
- [11] europe-west1
- [12] europe-west2
- [13] europe-west3
- [14] europe-west4
- [15] europe-west6
- [16] northamerica-northeast1
- [17] southamerica-east1
- [18] us-central1
- [19] us-east1
- [20] us-east4
- [21] us-west1
- [22] us-west2
- [23] us-west3
- [24] us-west4
- [25] cancel

Please enter your numeric choice: <select your numeric region choice>

To make this the default region, run `gcloud config set run/region REGION`.

Allow unauthenticated invocations to [SVC_NAME] (y/N)? y

Building using Buildpacks and deploying container to Cloud Run service [SVC_NAME] in project [PROJECT_ID] region [REGION]

✓ Building and deploying... Done.

✓ Uploading sources...

✓ Building Container... Logs are available at

```
[https://console.cloud.google.com/cloud-build/builds/BUILD-HASH?project=PROJECT_NUM].  
  ✓ Creating Revision...  
  ✓ Routing traffic...  
Done.  
Service [SVC_NAME] revision [SVC_NAME-00014-soc] has been deployed and is serving 100  
percent of traffic.  
Service URL: https://SVC_NAME-HASH-REG_ABBR.a.run.app
```

使用瀏覽器前往指定網址，確認部署作業是否成功！

如 `gcloud` 指令所示，使用者可以設定各種預設設定，減少輸出內容和互動，如上所示。舉例來說，如要避免所有互動，可以改用下列單行部署指令：

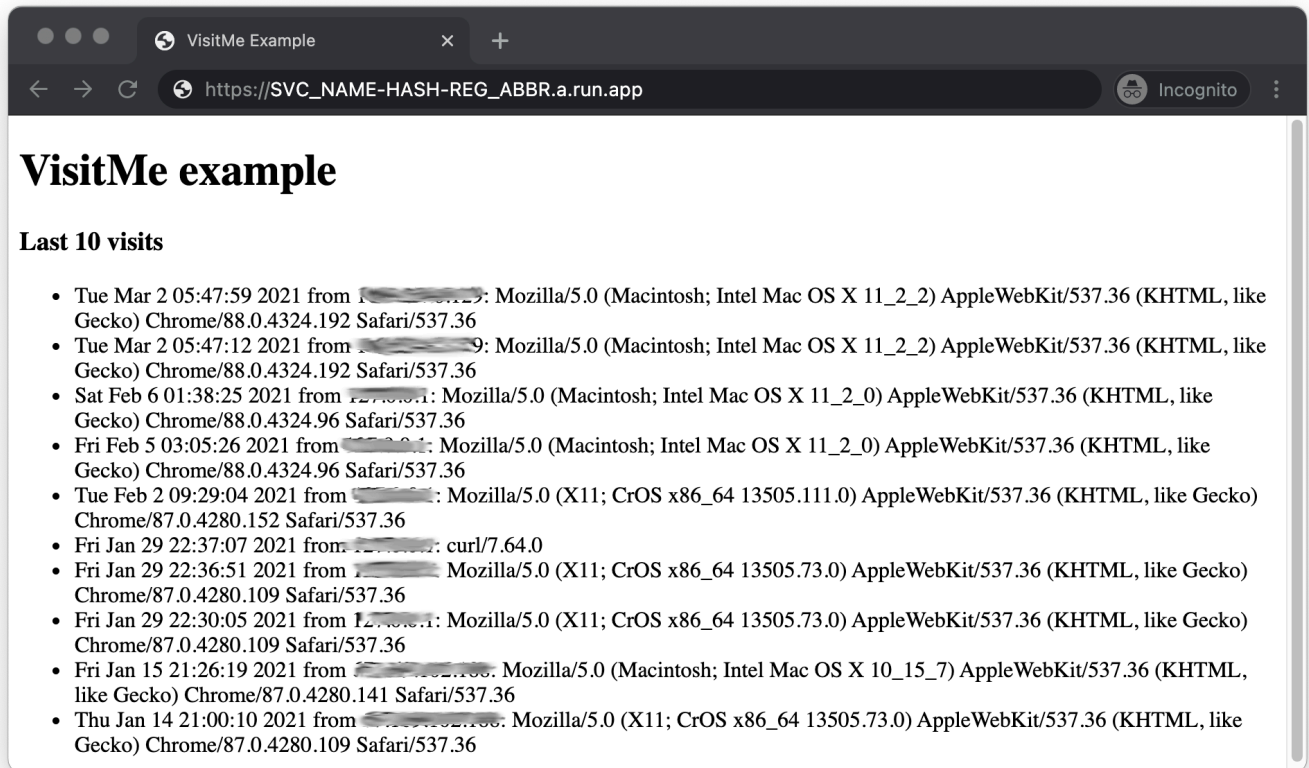
```
$ gcloud beta run deploy SVC_NAME --source . --platform managed --region REGION --allow-  
unauthenticated
```

如果您使用這個選項，請務必選取相同的服務名稱 `SVC_NAME` 和所需的 `REGION` 名稱，而不是如上述互動式操作般選取已建立索引的選單。

步驟 6 · 約 2 分鐘

6. 摘要/清除

確認應用程式在 Cloud Run 上的運作方式與在 App Engine 上相同。如果您直接跳到本系列課程，而未完成任何先前的程式碼研究室，應用程式本身不會變更，但會記錄所有對主要網頁 (/) 的造訪，且在您造訪網站足夠次數後，會顯示如下：



現在程式碼應與[第 5 堂課的存放區資料夾](#)內容相符。恭喜完成本單元 5 程式碼研究室。

選用：清除

在準備進行下一個遷移程式碼研究室之前，請先清除資源，以免產生帳單費用。由於您現在使用其他產品，請務必參閱[Cloud Run 定價指南](#)。

選用：停用服務

如果還沒準備好進行下一個教學課程，請[停用服務](#)，以免產生額外費用。準備好進行下一個程式碼研究室時，可以重新啟用這項功能。應用程式停用後，不會產生任何流量，因此不會產生費用。不過，如果[資料儲存空間用量](#)超出[免費配額](#)，您仍須支付相關費用，因此請刪除足夠的資料，確保用量低於上限。

另一方面，如果您不打算繼續遷移，並想完全刪除所有內容，可以[刪除服務](#)或[完全關閉專案](#)。

後續步驟

恭喜，您已將應用程式容器化，本教學課程到此結束！接下來，您可以前往第 4 模組程式碼研究室 (下方連結)，瞭解如何使用 Docker 執行相同操作，或是完成其他 App Engine 遷移作業：

- [單元 4](#)：使用 Docker 遷移至 Cloud Run
 - 使用 Docker 將應用程式容器化，以便在 Cloud Run 上執行
 - 可讓您繼續使用 Python 2

- **單元 7：** App Engine 發送工作佇列 (如果您使用 [發送] 工作佇列，則為必填)
 - 將 App Engine `taskqueue` 推送工作新增至模組 1 應用程式
 - 為第 8 單元中遷移至 **Cloud Tasks** 的作業做好準備
 - **單元 3：**
 - 將 Cloud NDB 的 Datastore 存取權現代化為 Cloud Datastore
 - 這是用於 Python 3 App Engine 應用程式和非 App Engine 應用程式的程式庫
 - **單元 6：** 遷移至 Cloud Firestore
 - 遷移至 Cloud Firestore，存取 Firebase 功能
 - 雖然 Cloud Firestore 支援 Python 2，但本程式碼研究室僅提供 Python 3 版本。
-

7. 其他資源

App Engine 遷移模組程式碼研究室問題/意見回饋

如果發現本程式碼研究室有任何問題，請先搜尋問題，再提出回報。搜尋及建立新問題的連結：

- App Engine 遷移程式碼研究室的問題追蹤器
 - [搜尋問題](#)
 - [回報新問題/提供意見](#)

遷移資源

下表提供第 2 和第 3 模組 (START) 以及第 5 模組 (FINISH) 的存放區資料夾連結。您也可以從[所有 App Engine Codelab 遷移作業的存放區](#)存取這些範例，並複製或下載 ZIP 檔案。

Codelab	Python 2	Python 3
Module 2	code	(代碼)
Module 3	(代碼)	code
模組 5	(不適用)	code

App Engine 和 Cloud Run 資源

以下是與這項特定遷移作業相關的其他資源：

- 容器
 - [Google Cloud Run](#)
 - [Google Cloud Build](#)
 - [Google Cloud Container Registry](#)
 - [Docker](#)
 - [Cloud Buildpacks 發布文章](#)
 - [Cloud Buildpacks 部署後作業](#)
 - [Cloud Buildpacks 存放區](#)
 - [CNCF Buildpacks 規格](#)
 - [App Engine 轉移至 Cloud Run 工具](#)：產生專屬 `service.yaml`
- 一般
 - [在 Google Cloud Platform 上執行 Python](#)
 - [Google Cloud Python 用戶端程式庫](#)